



Boletín 02 Intermedio — Primeros Pasos con Java

13 ejercicios de nivel intermedio en Java 21 / 25 LTS sobre métodos de String, funciones de Math, expresiones switch con yield, enum, fechas con java.time, desbordamiento de tipos y geometría computacional.

Este boletín contiene **13 ejercicios prácticos aplicados** para consolidar el manejo avanzado de tipos primitivos, la inmutabilidad y los métodos de búsqueda de la clase `String`, la precisión matemática con `Math`, los tipos enumerados `enum`, la manipulación de fechas con `java.time`, el control riguroso de memoria y la resolución de problemas de geometría computacional.

Antes de consultar el [Boletín Intermedio Resuelto](#), programa y verifica cada solución de manera autónoma en **IntelliJ IDEA**.

Ejercicio 1: Desglose Óptimo de Billetes y Monedas (Cajero Automático)

Diseña un programa que simule la dispensación de efectivo de un cajero automático. El usuario introduce una cantidad entera en euros (por ejemplo, `488 €`).

El programa debe calcular y mostrar el **mínimo número de billetes y monedas** necesarios para satisfacer el importe, utilizando las siguientes denominaciones de curso legal:

- Billetes: `50 €`, `20 €`, `10 €` y `5 €`.
- Monedas: `2 €` y `1 €`.

Solo deben mostrarse aquellas denominaciones cuya cantidad sea estrictamente superior a cero (`> 0`).

Ejercicio 2: El Envasado de Huevos de la Granja Avícola

El encargado de una granja avícola necesita un programa para clasificar los huevos recolectados en el día por sus gallinas.

Sabiendo que dispone de:

- **Envases gigantes (gruesas):** Capacidad para `144` huevos (12 docenas).
- **Envases medianos (docenas):** Capacidad para `12` huevos.
- **Envases pequeños (medias docenas):** Capacidad para `6` huevos.

Escribe un programa que solicite el total de huevos puestos hoy (por ejemplo, `308`) y determine:

1. Cuántos envases grandes se llenan completamente.
2. Cuántos envases de una docena se llenan con los huevos restantes.
3. Cuántos envases de media docena se llenan con el residuo.
4. Cuántos huevos quedan sueltos sin envasar.

Ejercicio 3: Teorema de Pitágoras con Validación Dinámica

Escribe un programa que calcule la hipotenusa de un triángulo rectángulo dados los dos catetos `a` y `b`.

1. Solicita ambos catetos en metros por teclado.
2. Calcula la hipotenusa de dos formas distintas:
 - Mediante `Math.sqrt(Math.pow(a, 2) + Math.pow(b, 2))`.
 - Mediante la función especializada de la biblioteca estándar: `Math.hypot(a, b)`.
3. Muestra el resultado formateado con **3 decimales de precisión** (`%.3f`).
4. Si alguno de los catetos introducidos es `≤ 0`, advierte al usuario de que las longitudes geométricas no pueden ser nulas ni negativas.

Ejercicio 4: Analizador y Parser de Nombres y Apellidos con `String`

Escribe un programa que solicite al usuario que teclee su nombre de pila y sus dos apellidos en una sola línea separados por espacios (sin partículas intermedias, por ejemplo: `Ada Lovelace Byron`).

El programa debe procesar la cadena utilizando exclusivamente métodos de la clase `String` (`indexOf`, `lastIndexOf`, `substring`, `length`, `toUpperCase`):

1. Aislar y mostrar el **nombre de pila** en mayúsculas y su número de letras.
2. Aislar y mostrar el **primer apellido** en mayúsculas y su número de letras.
3. Aislar y mostrar el **segundo apellido** en mayúsculas y su número de letras.
4. Generar y mostrar las **tres iniciales** unidas (ejemplo: `ALB`).

Ejercicio 5: Conversor de Potencia de Motores (CV a kW)

En la ficha técnica de los automóviles, la potencia del motor suele expresarse en **Caballos de Vapor (CV)** o en **Kilovatios (kW)**. La equivalencia física oficial es:

- `1 CV = 735.49875 W` (`0.73549875 kW`)
- `1 kW = 1.35962 CV`

Escribe un programa que ofrezca al usuario un menú de opciones:

1. Convertir de CV a kW.
2. Convertir de kW a CV.
3. Salir.

Muestra los resultados formateados con exactamente 2 decimales y advierte si la potencia introducida es menor o igual a cero.

Ejercicio 6: Tarificación Eléctrica con Tipos Enumerados (`enum`)

Crea una enumeración llamada `TramoHorario` con tres constantes: `VALLE`, `LLANO` y `PUNTA`. Las tarifas de la energía son:

- `VALLE`: 0.11 € / kWh
- `LLANO`: 0.18 € / kWh
- `PUNTA`: 0.28 € / kWh

Escribe un programa que solicite al usuario el número de kWh consumidos (valor decimal) y el tramo horario (como texto `VALLE`, `LLANO` o `PUNTA`).

1. Convierte el texto al valor correspondiente de la enumeración usando `TramoHorario.valueOf(texto.toUpperCase())`.
2. Calcula el coste de energía mediante un `switch`.
3. Aplica un **Impuesto sobre la Electricidad del 5.11%** y posteriormente el **IVA del 21%** sobre el total acumulado. Muestra la factura detallada.

Ejercicio 7: Clasificador de Caracteres Unicode y Métodos de `Character`

Escribe un programa que solicite al usuario pulsar una única tecla (leer un `char`). Utilizando los métodos estáticos de la clase `java.lang.Character` (`isDigit`, `isLetter`, `isUpperCase`, `isLowerCase`, `isWhitespace`), el programa debe determinar:

1. Si el carácter es un **dígito numérico** (del `'0'` al `'9'`) y mostrar su valor numérico entero.
2. Si es una **letra del alfabeto**, si está en mayúscula o minúscula, y si se trata de una **vocal** (`a`, `e`, `i`, `o`, `u`, incluidas mayúsculas y con tilde) o una **consonante**.
3. Si es un **espacio en blanco** o tabulador.
4. Si es un **símbolo o signo de puntuación**.
5. Muestra adicionalmente el **código Unicode / ASCII** numérico del carácter introducido.

Ejercicio 8: Evaluación de Expediente con Expresión `switch` y `yield`

Escribe un programa que reciba una nota numérica entera del 1 al 10 e implemente una expresión `switch` moderna que utilice la palabra clave `yield` para calcular un mensaje de mención académica personalizada:

- Notas 1 a 4: `yield "Insuficiente - Debe presentarse a la convocatoria extraordinaria"`.
- Notas 5 a 6: `yield "Suficiente / Bien - Supera los mínimos exigidos en el módulo"`.
- Notas 7 a 8: `yield "Notable - Demuestra un sólido dominio técnico de Java"`.
- Notas 9 a 10: `yield "Sobresaliente - Candidato a Matrícula de Honor"`.
- Cualquier otra nota: `yield "Nota no válida"`.

Ejercicio 9: Calculadora de Edad Exacta con `java.time`

Escribe un programa que solicite al usuario su fecha de nacimiento introduciendo por separado el **año**, el **mes** (1-12) y el **día**.

1. Construye un objeto `LocalDate` con la fecha indicada.
2. Obtén la fecha actual del sistema mediante `LocalDate.now()`.
3. Utiliza la clase `java.time.Period` (`Period.between(nacimiento, hoy)`) para calcular:
 - Años cumplidos.
 - Meses transcurridos desde el último cumpleaños.
 - Días transcurridos desde el último cumpleaños.
4. Si hoy coincide en día y mes con la fecha de nacimiento, el programa debe felicitar al usuario con un mensaje especial: *"¡Felicidades, hoy es tu cumpleaños!"*.

Ejercicio 10: Laboratorio de Desbordamiento Numérico (Overflow) y Casting

Escribe un programa experimental en Java que demuestre de forma tangible los peligros del estrechamiento (*narrowing casting*) y el desbordamiento de bits:

1. Asigna el valor `130` a una variable `int` y fuérla mediante cast a un `byte`. Imprime el resultado y explica por qué se convierte en un número negativo (`-126`).
2. Multiplica dos números enteros grandes: `int millon = 1_000_000; int resultado = millon * millon;`. Imprime el resultado y explica por qué no da un billón sino un número erróneo por desbordamiento de 32 bits. Muestra cómo solucionarlo realizando la operación con tipo `long`.

Ejercicio 11: Posición Relativa de Dos Circunferencias en el Plano

Escribe un programa en Java que solicite al usuario los parámetros geométricos de dos circunferencias en un plano cartesiano bidimensional:

- Circunferencia 1: Centro `(x1, y1)` y radio `r1`.
- Circunferencia 2: Centro `(x2, y2)` y radio `r2`.

El programa debe verificar que ambos radios sean estrictamente positivos (`r1 > 0` y `r2 > 0`), calcular la distancia euclídea entre sus centros: `d = √((x2 - x1)² + (y2 - y1)²)` y determinar su posición geométrica relativa según los siguientes criterios:

1. **Concéntricas:** `d == 0` y `r1 == r2` (coincidentes) o `d == 0` con radios distintos.
2. **Exteriores:** `d > r1 + r2`.
3. **Tangentes exteriores:** `d == r1 + r2`.
4. **Secantes:** `|r1 - r2| < d < r1 + r2`.
5. **Tangentes interiores:** `d == |r1 - r2|` con `r1 ≠ r2`.
6. **Interiores no concéntricas:** `d < |r1 - r2|`.

Ejercicio 12: Validador Integral de Fechas con Año Bisiesto Gregoriano

Diseña un validador riguroso de fechas que solicite tres números enteros correspondientes al **día**, **mes** y **año**.

El programa debe determinar si la combinación introducida conforma una fecha real y válida según las siguientes reglas:

1. El año debe ser mayor que cero (`año > 0`).
2. El mes debe estar comprendido entre `1` y `12`.
3. Si el mes es `1, 3, 5, 7, 8, 10` o `12`, el día debe estar entre `1` y `31`.
4. Si el mes es `4, 6, 9` o `11`, el día debe estar entre `1` y `30`.
5. Si el mes es `2` (febrero):
 - Comprobar si el año es bisiesto: `(año % 4 == 0 && año % 100 != 0) || (año % 400 == 0)`.
 - Si es bisiesto, febrero tiene hasta `29` días.
 - Si no es bisiesto, febrero tiene hasta `28` días.

Si la fecha no es válida, el programa debe informar con precisión del motivo exacto del rechazo (mes incorrecto, día fuera de rango para ese mes o intento de 29 de febrero en año no bisiesto).

Ejercicio 13: Ecuación de la Recta entre Dos Puntos y Tabla de Valores Tabulada

Escribe un programa que solicite las coordenadas de dos puntos en el plano cartesiano: `P1(x1, y1)` y `P2(x2, y2)` con valores decimales.

El programa debe:

1. Comprobar si `x1 == x2`. En tal caso, informar de que la recta es vertical (`x = x1`) y su pendiente es infinita, no pudiéndose expresar en la forma explícita habitual.
2. En caso contrario, calcular la **pendiente** `m` y la **ordenada en el origen** `n` de la ecuación explícita `y = m · x + n`:
 - o `m = (y2 - y1) / (x2 - x1)`
 - o `n = y1 - (m × x1)`
3. Mostrar por pantalla la ecuación resultante con 3 cifras decimales.
4. Generar una tabla de valores tabulada y alineada con `printf` evaluando la función en los siguientes 5 puntos: `x = -1.0`, `x = -0.5`, `x = 0.0`, `x = 0.5`, `x = 1.0`.